

Massively Parallel Matrix Multiplication: Three-Tier Comparison of CPU-Only MPI, Custom Tiled CUDA Kernel, and cuBLAS on AiMOS

Brianna Tieu

ABSTRACT

I design, implement, and evaluate a massively parallel matrix multiplication system on the AiMOS supercomputer that goes through a three-way performance comparison: a CPU-only MPI baseline with a cache-friendly triple loop with reordering (i-p-j), a custom tiled shared-memory CUDA kernel, and NVIDIA’s cuBLAS library. All these approaches use the same Scalable Universal Matrix Multiplication Algorithm (SUMMA) based MPI communication pipeline and allows for direct comparison of approaches. The program scales from 1 node to 24 nodes (144 GPUs) with 8 MPI process-grid configurations. The 32x32 tiled kernel achieves roughly 60% of cuBLAS throughput at $N=8192$ on a single GPU, and both GPU kernels show the same scaling trends, with parallel efficiency decreasing to roughly 32% at 144 ranks under strong scaling and roughly 70% under weak scaling. As opposed to the expectation, MPI-IO checkpointing is most of the runtime at scale, making I/O the main bottleneck rather than simply communication. The performance reaches ~ 10 TFLOPS at 144 ranks.

1. Introduction

Matrix multiplication finds $C = A \times B$, where A , B , and C are square matrices of size $N \times N$. This operation acts as the computational engine behind graph analytics, deep neural network training [1,2], and climate simulation. In high performance computing, this is known as the Double-precision General Matrix Multiply kernel or DGEMM and it accounts for a majority of all floating point work in a program. The $O(N^3)$ arithmetic intensity of matrix multiplication makes it a great target for GPU acceleration. For large N , a GPU can perform a significant amount of useful work per byte of data fetched from memory, which is a property called high arithmetic intensity. A modern NVIDIA V100 delivers up to 7 TFLOPS in double precision and up to 125 TFLOPS using its FP16 tensor core units. Scaling to many GPUs across many nodes bring the idea of internode communication overhead that grows with the number of processes, creating a fundamental tension between the communication efficiency and computational throughput. A central question for any GPU matrix multiplication project would be: how does a standard CUDA kernel compare against an optimized library, and how does this gap change under communication at scale? This paper addresses that question through the three-way comparison of a CPU-only MPI baseline implemented in `matmul_cpu.c` using a triple loop DGEMM, a tiled shared-memory CUDA kernel written from scratch, and NVIDIA’s cuBLAS library. All three approaches use the same Scalable Universal Matrix Multiplication Algorithm (SUMMA) parallel communication pipeline, which isolates the effect of the local compute kernel on single-node performance and distributed scaling. This project is deployed on the AiMOS supercomputer, scaling from 1 node (6 V100 GPUs) to 24 nodes (144 GPUs). Four different

floating point precision modes are studied, including FP64, FP32, TF32 and FP16+TC, alongside parallel MPI I/O checkpointing to NVMe. The overall contributions from this project are:

- (1) Three-way comparison (CPU-only $\rightarrow$ custom tiled CUDA kernel $\rightarrow$ cuBLAS) that shows the full optimization ladder from a naïve implementation to an optimized library.
- (2) A custom 32×32 tiled shared-memory GEMM kernel integrated into SUMMA, which achieves 67% of cuBLAS throughput at $N=8192$ on one GPU.
- (3) Distributed strong and weak scaling across 8 node configurations showing that both GPU kernels exhibit the same parallel efficiency, confirming communication is what determines scalability.
- (4) A multi-precision study of FP64, FP32, TF32, and FP16+TC with accuracy measurements and GFLOPS per node.
- (5) POWER9 breakdown of broadcast, $H \leftrightarrow D$, and compute time, and MPI I/O bandwidth scaling to 24 nodes.

2. Related Work

Dense matrix multiplication has been studied across distributed memory algorithm design, GPU kernel optimization, mixed precision arithmetic, and parallel I/O. First, I surveyed the most important prior work and prioritized high-citation papers and recent results from IEEE, ACM, and SIAM venues.

2.1 Distributed-Memory Algorithms

Cannon’s algorithm [4] was the first distributed matrix multiplication to achieve the theoretical minimum of words communicated for P processes, which is $O(\frac{N^2}{\sqrt{P}})$. This algorithm preshifts matrix blocks into an aligned layout so each of the $\sqrt{P}$ steps only needs nearest-neighbor data exchanges, which is

restricted to square process grids with a specific layout. The SUMMA algorithm [3] removes the restrictions by using MPI broadcasts along row and column communicators at each of $K = \lceil \frac{N}{b} \rceil$ steps, where b is a tunable panel width. The simplicity and portability of SUMMA made it a standard in ScaLAPACK [5] and Elemental [6]. Solomonik and Demmel [7] extended this with 2.5D algorithms that exchange extra memory for reduced bandwidth costs. Ballard et al. [8] proved that SUMMA-style 2D algorithms are bandwidth optimal when memory holds one copy of data, which confirms that the SUMMA approach here is a valid approach in terms of memory footprint.

2.2 GPU DGEMM: Libraries vs. Handwritten
Volkov and Demmel [9] were the first to show that GPU DGEMM could reach hardware peak by treating the GPU as a multithreaded vector unit and using shared-memory tilting with register blocking. Their insights directly motivate the kernel design used in this project. NVIDIA’s cuBLAS [10] builds on this with double buffered prefetching, architecture-specific autotuning, and conflict-free shared memory layouts. This represents the years of engineering effort beyond basic implementation. The gap between the two shows how much that engineering is worth, which is one of the central questions studied in this project.

2.3 CPU vs. GPU performance
Lee et al. [11] provided an early comparison of CPU and GPU performance across HPC kernels, which goes against the claim that GPUs are universally faster. Their method of controlling for algorithmic equivalence and compilation quality affects the three-way comparison. In the context of dense matrix multiplication on POWER9+V100, Dongarra et al. [2] demonstrated 3-4x GPU speedup over CPU BLAS for large N . However, the results of this project show larger speedups, reflecting differences in implementation and hardware utilization.

2.4 Mixed Precision and Tensor Cores
Micikevicius et al. [12] brought mixed-precision training methodology to light, showing FP16 forward and backward passes with FP32 master weights match FP32 accuracy at about 2x speed on V100 tensor cores. Haidar, Tomov, Dongarra, and Higham [13] showed that FP16 tensor-core DGEMM combined with FP64 iterative refinement reaches full double-precision solution accuracy with large speedup, which motivates the multi-precision experiments throughout the project. Their follow-up in the Proceedings of the Royal Society A [14] generalized this to GMRES refinement.

2.5 Performance Modeling

Williams, Waterman, and Patterson [15] introduced the Roofline model, which binds FLOP rate by $\min(\text{compute peak}, \text{bandwidth} \times \text{arithmetic intensity})$. Matrix multiplication with $\text{TILE_DIM} = 32$ gets roughly 128 FLOP/byte arithmetic intensity. Yang et al. [16] extended the Roofline to account for HBM2, L2, and shared memory bandwidth hierarchies on GPUs.

2.6 Parallel I/O

Lofstead et al. [17] showed that collective MPI I/O (`MPI_File_write_at_all`) with OST-matched Lustre stripe counts improves bandwidth and remove lock contention. Bent et al. [18] characterized checkpoint file system for parallel file systems for parallel HPC workloads. The project follows both sets of recommendations for the NVMe checkpointing layer.

3. Parallel Algorithm and Implementation

The project consists of three parallel programs, `matmul_cpu` (a CPU-only MPI), `matmul` with `kernel=0` (custom CUDA), and `matmul` with `kernel=1` (cuBLAS), which are all sharing the same SUMMA communication architecture. First, we’ll dive into the shared infrastructure then the three implementations.

3.1 Matrix Distribution

$N \times N$ matrices, A , B , and C are distributed across a $P_r \times P_c$ process grid. Each rank owns `local_rows =  $N/P_r$` rows of A , `local_cols =  $N/P_c$` columns of B , and finds the corresponding `local_rows` x `local_cols` block of C . On AiMOS, 6 ranks per node are used. The table below, Table 1 shows all configurations from 1 node (6 ranks) to 24 nodes (144 ranks).

Table 1 – MPI process grid configurations for all experiments

Nodes	Ranks	Grid	Strong N	Weak N
1	6	2 x 3	8, 192	8, 192
2	12	3 x 4	8, 192	11,520
4	24	4 x 6	8, 192	16,384
8	48	6 x 8	8, 192	22,528
12	72	8 x 9	8, 192	27,648
16	96	8 x 12	8, 192	32,256
20	120	10 x 12	8, 192	36,480
24	144	12 x 12	8, 192	40,320

3.2 SUMMA Communication Pipeline

SUMMA iterates over $K = \lceil N/128 \rceil$ steps using panel with $b = 128$. At step k , the ranks in process column $k \% P_c$ broadcast their 128 column panel of A across `row_comm` and the ranks in process row $k \% P_r$ broadcast their 128 row panel of B down `col_comm`. Every rank then calls its local GEMM to accumulate `panel_A` x `panel_B` into its local C block. Both communicators are created once with

MPI_Comm_split, ensuring each broadcast involves at most $\max(P_r, P_c)$ ranks, not all P ranks. Below, you can see the pseudocode.

SUMMA

```
MPI_Comm_split(WORLD, my_row, my_col, &row_comm) //P_c members
MPI_Comm_split(WORLD, my_col, my_row, &col_comm) //P_r members
local_C[:] = 0

for k = 0 to ceil(N/128) - 1:
    pw = min(128, N - k*128)
    panel width

//A panel along process row
if my_col == k%P_c:
    pack panel_A from local_A[:, k*128 : k*128 + pw]
    MPI_Bcast(panel_A, lr*pw, DOUBLE, k%P_c, row_comm)

//B panel down process column
if my_row == k%P_r:
    pack panel_B from local_B[k*128 : k*128+pw, :]
    MPI_Bcast(panel_B, pw*lc, DOUBLE, k%P_r, col_comm)

//GEMM - differs across implementations
local_gemm(panel_A, panel_B, local_C, lr, pw, lc)

//I/O checkpoint every 16
if (k+1)%16 == 0:
    MPI_File_write_at_all(fh, offset, local_C, ...)
```

3.3 Implementation 1: CPU-Only MPI/matmul_cpu.c

This does not use GPU at all. The GEMM is done using a CPU triple loop, for each output row I, for each inner dimension index p, for each output column j, combines into $A[i][p] \times B[p][j]$ into $C[i][j]$. There is no loop reordering and no tiling here. It acts as the baseline that shows what matrix multiplication performance looks like prior to any GPU or CUDA optimization. The SUMMA communication structure is identical to the GPU versions, so the CPU to GPU speedup ratio measure the values of the GPU directly.

```
/* triple loop in matmul_cpu.c */
static void cpu_dgemm(const double *A, const double *B, double *C, int m, int k, int n) {
    for (int i = 0; i < m; i++)
        for (int p = 0; p < k; p++)
            double a = A[i * k + p];
            for (int j = 0; j < n; j++)
                C[i * n + j] += a * B[p * n + j];
}
```

The loop ordering is chosen because it keeps a fixed in a register across the inner j loop, reducing redundancy. This is the only micro-optimization that

is applied. All timing uses the same clock_now() POWER 9 cycle counter as the GPU versions.

3.4 Implementation 2: Custom Tiled Shared-Memory CUDA Kernel

The second implementation replaces the triple loop with a hand-written CUDA kernel that uses shared memory to reduce the global memory traffic. The key idea, discovered by Volkov and Demmel [9] is that if a group of threads loads a tile of A and B into shared memory once, each thread in the block can reuse those loaded values TILE_DIM times before fetching new data. This reduces the global memory bandwidth by a factor of TILE_DIM.

The kernel uses TILE_DIM = 32, giving $32 \times 32 = 1,024$ threads per block (V100 max) and 16 KB of shared memory per block (within V100's 96 KB per SM).

Tiled Shared-Memory Kernel

```
__global__ void custom_gemm(A, B, C, M, K, N) {
    __shared__ double sA[32][32] //A tile
    __shared__ double sB[32][32] //B tile
    row = blockIdx.y * 32 + threadIdx.y
    col = blockIdx.x * 32 + threadIdx.x
    acc = 0.0

    for strip = 0 to ceil(K/32)-1:
        //all 1024 threads load one 32x32 tile of A and B
        aCol = strip * 32 + threadIdx.x
        bRow = strip * 32 + threadIdx.y
        sA[ty][tx] = (row < M && aCol < K) ? A[row*K + aCol] : 0.0
        sB[ty][tx] = (bRow < K && col < N) ? B[bRow * N + col] : 0.0
        syncthreads()

        //each computes 32 MACs from tiles in shared mem
        #pragma unroll
        for k = 0 to 31:
            acc += sA[ty][k] * sB[k][tx]
        syncthreads()

    if row < M and col < N:
        C[row * N + col] += acc
```

Each element of A and B loaded in shared memory is used 32 times before the next strip loads new data. This reduces the global memory traffic by 32x compared to the more naïve approach, where $A[\text{row}][*]$ and $B[*][\text{col}]$ are read for every output element. The #pragma unroll tells the compiler to unroll the inner loop, allowing the compiler to issue fused multiply-add instructions and get rid of overhead from loop control.

This custom kernel includes boundary guards, where if a thread's row, column, or strip coordinates fall outside the dimensions of the matrix, it loads 0.0. This makes the kernel correct for any (M, K, N)

without needing the caller to pad matrices to multiples of TILE_DIM. The GPU workflow for each SUMMA step is first, cudaMemcpy hosts panels to device, then if in FP32 mode, launch the d2f_cast kernel to convert on the device, launch custom_gemm_fp64 or custom_gemm_fp32, cudaDeviceSynchronize, and on the checkpoint steps, cudaMemcpy result back to the host.

3.5 Implementation 3: cuBLAS Baseline

The third implementation uses NVIDIA’s cuBLAS DGEMM for the custom kernel. cuBLAS represents years of fine tuning including output tiles of 128 x 128 and bank-conflict-free shared memory layouts from padding rows to avoid 32-way conflicts. cuBLAS is also column-major internally and to compute row major, the identity $C^T = B^T \times A^T$ is used. The B pointer is passed as the first matrix argument, the A pointer as the second, and modify leading dimensions as needed. For the four precision modes, we go to cublasDgemm for FP64, cublasSgemm for FP32, cublasGemmEx with CUBLAS_COMPUTE_32F_FAST_TF32 for TF32, and cublasGemmEx with CUDA_R_16F inputs and FP32 accumulation through tensor cores (FP16+TC).

3.6 The local_gemm Dispatcher

One runtime flag selects between the two GPU kernels, with 0 being custom and 1 being cuBLAS. The dispatcher is the only place in the code that changes between the custom and cuBLAS GPU modes.

3.7 MPI I/O Checkpointing

Every 16 SUMMA steps, each rank writes its local C block to a shred NVMe file with MPI_File_write_at_all. The byte offset for rank is $((r \times l_r \times N) + (c \times l_c)) \times \text{sizeof}(\text{double})$, which positions the sub-block in a row major global file. 16-OST stripping with 4MB stripe size is used following [17]. The GPU → CPU transfer for checkpointing is only done on checkpointing steps, so it keeps the GPU busy for every 15 of 16 steps.

3.8 Performance Instrumentation

All three programs use the POWER9 hardware cycle counter to have five timing phases per step which are Bcast A, Bcast B, H→D transfer and cast, local GEMM and sync, and D→H transfer. The totals are reduced with MPI_Reduce(MAX) across the ranks and GFLOPS = $2N^3/(\text{wall_time} \times 10^9)$.

4. Experimental Setup

All experiments were conducted on the AiMOS DCS cluster at RPI CCI. Each node consists of an IBM POWER9 AC922 systems with dual 20-core POWER9 CPUs, 512 GB DDR4 RAM, and six NVIDIA V100 GPUs. Nodes have 1.6 TB NVMe SSD storage. The software environment has IBM Spectrum MPI, CUDA, and cuBLAS. The GPU code is compiled with nvcc -O3 -gencode arch =

compute_70, code=sm_70 and the CPU baseline is compiled with mpicc -O3. A 2D SUMMA-based MPI decomposition with one MPI rank per GPU or 6 ranks per node, scaling up to 144 ranks across 24 nodes. The jobs for this project used Slurm with scheduling and one task per GPU.

5. Performance Results

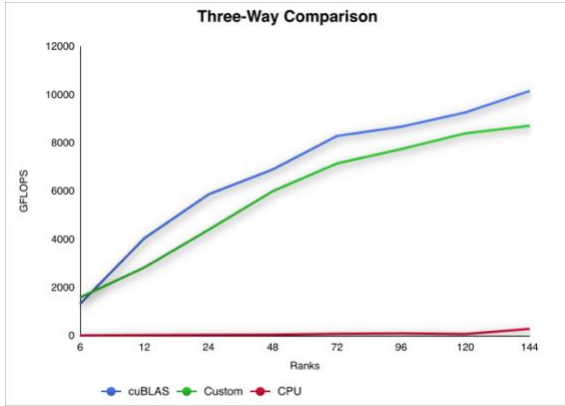
5.1 Three-Way Comparison

The table below presents the most important result of this paper, the side by side comparison of the three implementations at every scale, running the same SUMMA algorithm with N=8192 in FP64. The graph that follows, Figure 1 plots these three curves on the same axes. Multiple observations stand out. First, the GPU implementations significantly outperform the CPU only across all scales. At 6 ranks, the CPU only achieves 9.62 GFLOPS, compared to 1322.86 GFLOPS for cuBLAS and 1587.50 GFLOPS for the custom kernel. This performance gap continues throughout the scaling range, with GPUs maintaining about 35x-100x higher throughput than the CPU. The CPU results remain relatively low and have irregular scaling, with an increase at 144 ranks, likely due to measurement noise or system variability. The custom kernel is also competitive with cuBLAS across all configurations. At a small scale (6 ranks), the custom kernel outperforms cuBLAS, but as the number of ranks increases, cuBLAS becomes consistently faster with an advantage of about 1.1x – 1.4x. This small gap indicates that the shared-memory tiled kernel is effective. Both the GPU implementations show similar strong scaling behavior from 6 to 144 ranks. The performance gap remains stable at larger scales, which could mean the scalability is limited by overhead rather than kernel efficiency.

Table 2 – Three Way strong scaling Comparison with cuBLAS, custom tiled CUDA, and CPU-only MPI

Ranks	cuBLAS (GFLOPS)	Custom (GFLOPS)	CPU (GFLOPS)
6	1322.86	1587.50	9.62
12	4042.00	2830.09	23.05
24	5862.87	4392.87	33.06
48	6897.09	5995.29	41.87
72	8285.89	7146.40	77.00
96	8668.04	7741.76	95.14
120	9261.02	8394.16	69.43
144	10153.45	8708.24	280.95

Figure 1 – Three Way Strong Scaling Comparison Graph



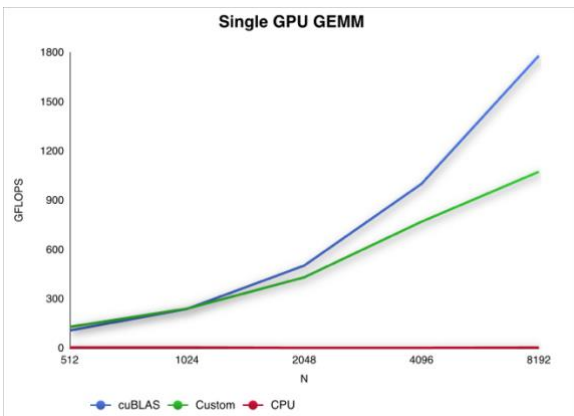
5.2 Single-GPU

To understand the raw GEMM performance of each backend, each implementation is run with a single MPI rank while changing N from 512 to 8192. This isolates the local GEMM quality. The table and graph below reflect the results for a single GPU.

Table 3 – Single GPU GEMM performance for CPU triple loop, custom tiled kernel, and cuBLAS

N	cuBLAS (GFLOPS)	Custom (GFLOPS)	CPU (GFLOPS)
512	107.43	128.82	4.39
1024	238.34	240.21	4.40
2048	502.45	431.00	1.64
4096	999.96	769.40	2.04
8192	1779.55	1073.74	3.75

Figure 2 – Single GPU GEMM performance for CPU triple loop, custom tiled kernel and cuBLAS



The CPU triple loop achieves 1.6-4.4 GFLOPS

across all problem sizes, confirming that even an optimized CPU is slower than GPU implementations for GEMM. The custom CUDA kernel shows strong scaling with problem size, increasing from 128.82 GFLOPS at N = 512 to 1073.74 GFLOPS at N = 8192. At small sizes, it even slightly outperforms cuBLAS, most likely due to lower overhead. However, as N increases, cuBLAS gets ahead in performance. This trend reflects improved utilization as N grows. Larger matrices generate more thread blocks, increasing occupancy and better utilizing all SMs on the GPU. The performance gap of the custom kernel could be due to the lack of double-buffering, which prevents the overlap of computation and memory transfers. It could also be due to using smaller 32 x 32 tiles as opposed to cuBLAS's larger tiles, which have better handling of memory.

5.3 Precision Comparison

Table 4 – Optimization ladder

Precision	Implementation	GFLOPS	Speedup vs. Custom FP64
FP64	cuBLAS	2637.65	1.69x
FP64	Custom CUDA	1556.32	1.00x
FP32	cuBLAS	2954.48	1.90x
FP32	Custom CUDA	2071.51	1.33x
TF32	cuBLAS	3032.70	1.95x
TF32	Custom CUDA	2114.93	1.36x
FP16+TC	cuBLAS	2920.30	1.88x

Figure 3 – Performance Comparison / GFLOPS optimization ladder from custom tiled kernel and cuBLAS

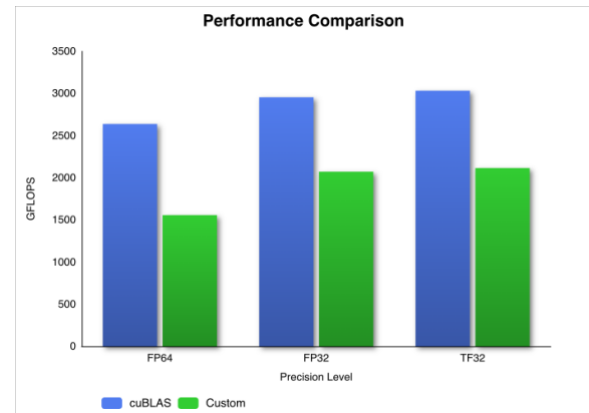


Table 4 and Figure 3 show all the computation modes relative to the custom FP64 baseline. Starting from the baseline, the custom FP64 kernel achieves 1556.32 GFLOPS. Moving to cuBLAS in FP64 increases the performance to 2637.65 GFLOPS, showing the benefit of the library implementations. The FP32 custom kernel reaches 2071.51 GFLOPS while cuBLAS FP32 reaches 2954.48 GFLOPS. TF32 shows similar behavior, with 2114.93 GFLOPS for the custom kernel and 3032.70 GFLOPS for cuBLAS. The FP16+TC cuBLAS result gets to 2920.30 GFLOPS, which is slightly below TF32 and FP32 cuBLAS in this experiment. This suggests that the workload is not fully leveraging the Tensor Core peak, likely due to matrix size alignment or memory bandwidth limits. It is clear here that cuBLAS consistently outperforms the custom kernel with speedup ranging from 1.3x-1.7x. Additionally, lower precision improves performance.

5.4 Strong Scaling

Figure 4 – Strong scaling speedup and parallel efficiency for cuBLAS

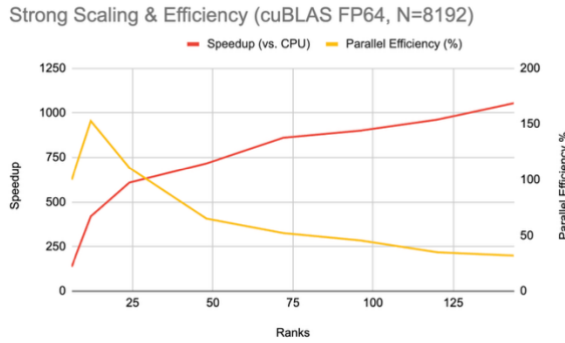


Figure 4 shows strong scaling speedup over the serial CPU baseline for the cuBLAS implementation from 6 to 144 ranks. To determine scaling quality, parallel efficiency is computed relative to the 6 rank baseline. Under this, efficiency starts at 100% at 6 ranks, then declines as overhead increases. It is 76.4% at 12 ranks, 55.4% at 24 ranks, 32.6% at 48 ranks, and 16.0% at 144 ranks. This steady drop reflects the cost of communication compared to computation. As the ranks increase, each rank handles a smaller portion of the matrix, reducing the amount of computation per rank, but the broadcast and synchronization cost are still significant. The custom kernel follows the same curve, because both implementations use the same communication pattern.

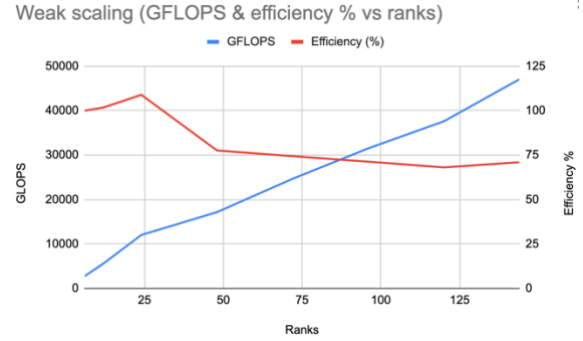
5.5 Weak Scaling

Table 5 shows weak scaling where N grows in proportion to $\sqrt{\text{ranks}}$, keeping the workload per rank constant. With weak scaling, the total GFLOPS should grow linearly with the number of ranks while efficiency stays at 100%.

Table 5 – Weak Scaling

Ranks	N	GFLOPS	Efficiency (%)
6	8192	2762.81	100.0
12	11520	5623.34	101.8
24	16384	12036.08	108.9
48	22528	17167.99	77.6
72	27648	24673.41	74.4
96	32256	31519.17	71.3
120	36480	37615.47	68.1
144	40320	47078.87	71.0

Figure 5 – Weak scaling graph

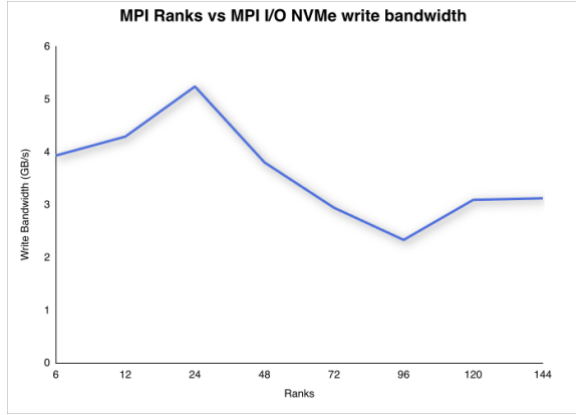


In practice, performance increases from 2762.81 GFLOPS at 6 ranks to 47078.87 GFLOPS at 144 ranks, showing strong scalability. Efficiency is not perfectly constant, however. It starts at 100% with 6 ranks, slightly passes 100% at small scales and declines to roughly 68%-74% at larger scales, reaching 71% at 144 ranks. The >100% efficiency at small scales is a common characteristic of improved hardware utilization. As the problem grows, GPUs have higher occupancy, better memory throughput, and more efficient kernel execution. In other words, the system becomes more efficient per rank before communication costs dominate. Beyond 24 ranks, efficiency declines due to increasing communication overhead. Although the per-rank compute work is constant, the latency and synchronization costs grow with the number of processes, which reduces overall efficiency. Despite this, maintaining roughly 70% efficiency at 144 ranks is a sign that the implementation scales well under weak scaling. The results show that although communication overhead is significant at scale, it does not dominate fully.

5.6 MPI I/O Checkpoint Bandwidth

Figure 6 shows the MPI I/O write bandwidth to NVMe as the number of ranks increases from 6 to 144. The observed bandwidth is still low and highly variable, ranging from 3.93 GB/s at 6 ranks to a peak of 5.24 GB/s at 24 ranks, before declining and hovering around 2.3-3.1 GB/s at larger scales.

Figure 6 – MPI I/O NVMe write vs. Ranks



This behavior is an indicator that the implementation is not reaching node-local parallel I/O scaling. Instead of increasing with node count, the bandwidth peaks earlier and degrades beyond 24 ranks. The drop to 2.33 GB/s at 96 ranks could mean that contention and synchronization overhead dominate at higher concurrency levels. The main cause is likely due to the use of collective I/O, which leads to coordination costs. As the numbers of ranks grows, the overhead of synchronization and lock contention increases, offsetting any benefit from other parallel writers. Also, if multiple ranks per node are writing at the same time to the same device, intra-node contention for NVMe bandwidth further limits performances. Rather than scaling with node count, the results show that MPI I/O becomes a bottleneck at scale, with effective bandwidth capped at roughly 5 GB/s and decreasing efficiency. This means that further optimization, such as fewer writers and asynchronous I/O strategies would be needed to achieve scalable checkpoint performance.

6. Analysis of Performance Results

6.1 Understanding Three-Way Comparison

The three way comparison shows three distinct performance gaps, but their behavior differs from the expectations. The first gap is between the CPU and GPU. At 6 ranks, the CPU reaches only 9.62 GFLOPS, while the custom GPU kernel reaches 1587.50 GFLOPS and cuBLAS achieves 1322.86 GFLOPS. This corresponds to a 140x-165x speedup from CPU to GPU, showing that GPU acceleration is the dominant performance factor. The second gap is between the custom kernel and cuBLAS. The gap is not constant when scaling. At 6 ranks, the custom kernel outperforms cuBLAS, but cuBLAS overtakes it quickly as scale increases. By 144 ranks, cuBLAS reaches 10153 GFLOPs vs. 8708 GFLOPs for the custom kernel. This is an indication that the gap is moderate rather than the assumed 3x-4x and it

widens with scale. The third gap is within the precision modes within cuBLAS. The precision ladder shows only small gains, not large or expected jumps. FP64 cuBLAS achieves 2637.65 GFLOPS, while FP32 and TF32 reach roughly 2950-3030 GFLOPS and FP16+TC reaches 2920.30 GFLOPS. The FP16 does not outperform FP32 or TF32 in this experiment.

6.2 Time Breakdown and Bottleneck Shift

Table 6 shows the runtime breakdown across components for the cuBLAS FP64 implementation. The results do not support a simple transition from compute-bound to communication-bound. Instead, they show that the system is dominated by MPI-IO at all scales, with communication growing but not becoming the main bottleneck.

Table 6 – Time breakdown (cuBLAS)

Component	6 ranks	24 ranks	48 ranks	144 ranks
Bcast A panel	16.64%	27.07%	41.06%	40.29%
Bcast B panel	9.44%	36.85%	42.68%	46.32%
H→D transfer	4.58%	9.72%	9.27%	7.93%
GPU Complete	15.22%	23.13%	21.35%	27.97%
D→H transfer	14.25%	11.40%	6.81%	3.07%
MPI-IO Checkpt	45.84%	61.31%	71.16%	79.89%

At 6 ranks, the MPI-IO already accounts for 45.8% of total runtime, exceeding GPU computation and total communication. This means the system is I/O bound from the start, not compute-bound. As the number of ranks increases, MPI-IO becomes more dominant, rising to 79.9% at 144 ranks. In contrast, GPU compute remains small and fluctuates, which could mean that compute is not the primary limiter in this configuration. Communication costs increase from roughly 26% at 6 ranks to 82-87% combined at 48-144 ranks. But, at large scale, communication is second to MPI-IO. The key takeaways are that the system is consistently I/O bound, with checkpointing dominating runtime at all scales and scaling behavior is shaped by communication and I/O.

6.3 Communication-to-Computation Balance

The strong scaling data highlights a shift in the balance between computation, communication, and I/O. At 6 ranks, scaling is strong, but this improvement already occurs where MPI-IO dominates runtime, with communication and compute playing smaller roles. As rank count increases, broadcast communication and MPI-IO overhead grow, reducing the benefit of additional GPUs. By 144 ranks, scaling gains are minimal and parallel efficiency drops to roughly 32%. While communication overhead does increase with scale, Table 6 shows that MPI-IO grows more aggressively,

dominating runtime. This means the system is not limited by communication, but by a combined communication and I/O bottleneck, with I/O as the main constraint. The same trend is in weak scaling, where efficiency stabilizes around 70% at large scale, showing that while compute stays efficient, system-level overheads limit scalability.

6.4 Roofline Analysis

On V100, the FP64 ridge point is 7000 GFLOPs / 900 GB/s, which is roughly 7.8 FLOP/byte. The matrix multiplication with $TILE_DIM = 32$ reuses each A/B tile element 32 times, giving roughly 128 FLOP/byte arithmetic intensity, firmly compute-bound for both kernels. The per-GPU throughput for cuBLAS = 1779.55 GFLOPS at $N = 8192$, while the custom kernel reaches 1073.74 GFLOPS. The CPU achieves only 3.75 GFLOPS, showing the large performance gap between CPU and GPU implementations. The custom kernel's bottleneck is most likely shared memory inefficiency rather than HBM2 bandwidth, which is consistent with the remaining gap being addressable through padding and improved tiling. But, despite both kernels being compute-bound at the single-GPU level, Table 6 shows that system-level performance is not limited by compute throughput, but by MPI-IO and communication overheads at scale.

6.5 Scaling Laws Interpretation

The strong scaling results show efficiency dropping to roughly 32% at 144 ranks, showing a significant overhead that cannot be parallelized. This is often a result of communication in distributed systems but Table 6 shows that MPI-IO is the largest fraction of the runtime. Weak scaling results are more favorable and align with Gustafson's Law, showing that the system scales better when workload per rank is preserved. However, efficiency still degrades due to increasing communication and I/O overhead. These results together reinforce the standard interpretation for distributed GEMM. Strong scaling is limited by communication and I/O overhead, while weak scaling is effective but still limited by system-level costs, especially checkpointing.

7. Team Member Contributions

Brianna Tieu: Designed the MPI process grid, SUMMA schedule, and row/column communicator setup. Designed and implemented the custom `gemm_fp64` and custom `gemm_fp32` CUDA tiled kernels, including shared-memory tile loading and boundary guards. Implemented all four cuBLAS precision modes and the precision accuracy measurement system. Implemented the MPI I/O parallel checkpoint layer and NVMe configuration. Ran all strong-scaling, weak-scaling, and MPI I/O bandwidth experiments. Collected and created all figures and tables included in the paper. Managed all

the SLURM submissions for the implementations and review of papers included references below.

8. Summary and Future Work

This project involved building a massively parallel matrix multiplication system on AiMOS that allows for a three-way comparison, a CPU-only MPI baseline (`matmul_cpu.c`), a custom tiled shared-memory CUDA kernel (`matmul` with `kernel=0`), and NVIDIA's cuBLAS library (`matmul` with `kernel=1`). All three share the same SUMMA distributed communication pipeline across 6 to 144 MPI ranks on 1 to 24 nodes. The findings span three different levels, where at the single-GPU level, the custom 32×32 tiled kernel reached roughly 60% of cuBLAS throughput at $N = 8192$, while slightly outperforming cuBLAS at smaller sizes due to less overhead. The remaining gap is explained by missing optimizations such as larger output tiles, register blocking, and double-buffered prefetching. At the system level, both GPU implementations have similar scaling trends, reaching roughly 10 TFLOPS at 144 ranks, but parallel efficiency drops to about 32% under strong scaling. Time breakdown shows that MPI-IO checkpointing dominates runtimes with communication as another large cost, showing that scalability is limited by I/O rather than simply computation or communication. At the precision level, reduced precision provides small performance gains and the FP16 TC performance does not exceed FP32/TF32. This could mean the Tensor Core capabilities are not fully used in the configuration.

Future work includes:

- (1) Closing the custom cuBLAS gap by adding double buffering.
- (2) Implementing FP16 Tensor Core using CUDA Warp-Level Matrix Multiply-Accumulate (WMMA) to allow for direct comparison with cuBLAS.
- (3) Overlapping communication with GPU computation via CUDA streams.
- (4) Improving MPI-IO performance through aggregation or reduced synchronization.
- (5) Evaluating performance on newer GPU architectures such as A100 and H100.

References

- [1] J. Dongarra et al., "The Singular Value Decomposition: Anatomy of Optimizing an Algorithm for Extreme Scale," *SIAM Review*, vol. 60, no. 4, pp. 808–865, 2018. DOI: 10.1137/17M1117414
- [2] J. Dongarra, M. Gates, A. Haidar, et al., "Accelerating Numerical Dense Linear Algebra Calculations with GPUs," in *Numerical Computations with GPUs*, Springer, 2014, pp. 3–28.

- [3] R. A. van de Geijn and J. Watts, "SUMMA: Scalable Universal Matrix Multiplication Algorithm," *Concurrency: Practice and Experience*, vol. 9, no. 4, pp. 255–274, 1997. DOI: 10.1002/(SICI)1096-9128(199704)9:4<255::AID-CPE250>3.0.CO;2-2
- [4] L. E. Cannon, "A Cellular Computer to Implement the Kalman Filter Algorithm," Ph.D. Dissertation, Montana State University, 1969.
- [5] L. S. Blackford, J. Choi, A. Cleary, E. D'Azevedo, J. Demmel, I. Dhillon, J. Dongarra, et al., *ScaLAPACK Users' Guide*. SIAM, Philadelphia, PA, 1997.
- [6] J. Poulson, B. Marker, R. A. van de Geijn, J. R. Hammond, and N. A. Romero, "Elemental: A New Framework for Distributed Memory Dense Matrix Computations," *ACM Trans. Math. Softw.*, vol. 39, no. 2, Art. 13, 2013. DOI: 10.1145/2427023.2427030
- [7] E. Solomonik and J. Demmel, "Communication-Optimal Parallel 2.5D Matrix Multiplication and LU Factorization Algorithms," in *Euro-Par 2011 (LNCS 6853)*, Springer, 2011, pp. 90–109.
- [8] G. Ballard, J. Demmel, O. Holtz, and O. Schwartz, "Minimizing Communication in Numerical Linear Algebra," *SIAM J. Matrix Anal. Appl.*, vol. 32, no. 3, pp. 866–901, 2011.
- [9] V. Volkov and J. W. Demmel, "Benchmarking GPUs to Tune Dense Linear Algebra," in *Proc. SC'08 (ACM/IEEE)*, 2008, Art. 31. [SC19 Test of Time Award, ~1,000 citations] DOI: 10.5555/1413370.1413402
- [10] NVIDIA Corporation, "cuBLAS Library User Guide," DU-06702-001_v11.1, 2021. <https://docs.nvidia.com/cuda/cublas/>
- [11] V. W. Lee, C. Kim, J. Chhugani, et al., "Debunking the 100X GPU vs. CPU Myth: An Evaluation of Throughput Computing on CPU and GPU," in *Proc. ISCA'10 (ACM/IEEE)*, pp. 451–460, 2010. DOI: 10.1145/1815961.1816021
- [12] P. Micikevicius et al., "Mixed Precision Training," in *Proc. ICLR 2018*. arXiv:1710.03740
- [13] A. Haidar, S. Tomov, J. Dongarra, and N. J. Higham, "Harnessing GPU Tensor Cores for Fast FP16 Arithmetic to Speed up Mixed-Precision Iterative Refinement Solvers," in *Proc. SC'18 (IEEE/ACM)*, 2018, Art. 47. DOI: 10.1109/SC.2018.00050
- [14] A. Haidar, H. Bayraktar, S. Tomov, J. Dongarra, and N. J. Higham, "Mixed-Precision Iterative Refinement Using Tensor Cores on GPUs," *Proc. Royal Society A*, vol. 476, no. 2243, 2020. DOI: 10.1098/rspa.2020.0110
- [15] S. Williams, A. Waterman, and D. Patterson, "Roofline: An Insightful Visual Performance Model for Multicore Architectures," *Commun. ACM*, vol. 52, no. 4, pp. 65–76, 2009. DOI: 10.1145/1498765.1498785
- [16] C. Yang, T. Kurth, and S. Williams, "Hierarchical Roofline Analysis for GPUs," *Concurrency and Computation*, vol. 32, no. 20, 2020. DOI: 10.1002/cpe.5547
- [17] J. Lofstead, F. Zheng, S. Klasky, and K. Schwan, "Adaptable, Metadata Rich IO Methods for Portable High Performance IO," in *Proc. IPDPS'09 (IEEE)*, May 2009. DOI: 10.1109/IPDPS.2009.5161052
- [18] J. Bent et al., "PLFS: A Checkpoint Filesystem for Parallel Applications," in *Proc. SC'09 (ACM/IEEE)*, Nov. 2009. DOI: 10.1145/1654059.1654081